
[TODO: Topic 1 / 2 / 3 / 4 – Title]

[TODO: Team Name]

[TODO: Member A] ▪ [TODO: Member B] ▪ [TODO: Member C]

Final Project Defense – [TODO: Date]

[TODO: [github.com/your-team/your-repo](#)]

The problem in one sentence

What the system does

[TODO: One sentence stating the user problem and the system's contract.

Example: “A user uploads a meal photo; we return calories, protein, carbs, and fat in under two seconds, citing the USDA entry for every ingredient.”]

Inputs

- ▶ **[TODO: e.g. meal image (JPEG/PNG, ≤ 5 MB)]**
- ▶ **[TODO: e.g. user profile]**

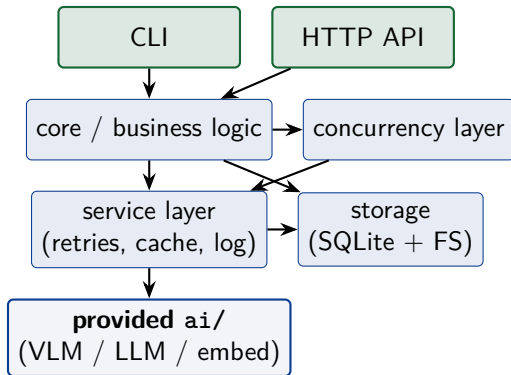
Outputs

- ▶ **[TODO: e.g. structured JSON with per-ingredient kcal/protein/carbs/fat]**
- ▶ **[TODO: e.g. history record in SQLite]**

Entry points: [TODO: CLI command + HTTP endpoint(s)]

Architecture

Module map. The boundary between the *provided ai/* package and *our* SE layer is shown explicitly.



Replace this generic diagram with one specific to your team's modules.

Three design decisions we can defend

1. [TODO: Decision 1 (e.g. “asyncio over threads”)]
[TODO: One-line justification + the alternative we rejected.]
2. [TODO: Decision 2 (e.g. “Pydantic Settings over manual env parsing”)]
[TODO: One-line justification + trade-off accepted.]
3. [TODO: Decision 3 (e.g. “SQLite + filesystem blobs over a single blob column”)]
[TODO: One-line justification + what would push us to change it.]

Expect to be asked

“Why did you not use **[TODO: the obvious alternative]**?” Have a 15-second answer ready for each of the three.

Concurrency: the benchmark

Workload: [TODO: e.g. “analyze 16 meals end-to-end on the developer laptop, fresh cache”]

	Sequential	Concurrent (sem=10)
Wall-clock (s)	[TODO: 42.1]	[TODO: 7.3]
Speedup	1.0×	[TODO: 5.8×
Provider 429s	0	0

Bottleneck named: [TODO: e.g. “the embedding endpoint is dominant; the per-request RTT is 70 ms”]

What didn't reach $N\times$: [TODO: e.g. “USDA's free tier caps at 1000 req/h, so we bound the semaphore at 10”]

Numbers must reproduce from the README. Have the exact command line ready.

Robustness: one concrete failure mode

Scenario: [TODO: e.g. “the LLM returned valid JSON with "meal_recognized": true but an empty ingredients list”]

How we surfaced it: [TODO: e.g. a property-based test that fed random malformed bytes; manual chaos test that pulled the network mid-call.]

What the system does:

- ▶ [TODO: Retry: tenacity with exponential backoff (3 attempts, 1–8s).]
- ▶ [TODO: Validate: structured “unknown meal” response, not a 500.]
- ▶ [TODO: Log: WARN ingredients_empty image=... duration=... with no PII.]
- ▶ [TODO: Degrade: HTTP returns 200 with an explicit recognized=false field.]

Live demo (or scripted demo)

What we will run

[TODO: One sentence describing what the live demo shows.]

Demo command (works inside Docker):

```
$ docker run --env-file .env finalproj \  
python scripts/demo.py --input data/sample_meal.jpg
```

- ▶ **[TODO: Bullet describing the most interesting on-screen output.]**
- ▶ **[TODO: Bullet calling out the timing improvement vs. sequential.]**

Have a pre-recorded screencast as backup in case the network is slow.

Testing

Coverage & shape

- ▶ Total coverage: **[TODO: 72] %**
- ▶ Provided ai/ smoke tests: **passing**
- ▶ Test count: **[TODO: 42 own + 28 smoke]**
- ▶ All tests offline; AI module fully mocked

Three tests worth calling out

- ▶ **[TODO: Happy-path end-to-end]**
- ▶ **[TODO: Concurrency: one task raises, gather degrades]**
- ▶ **[TODO: Error path: provider 429 surfaces as retry]**

Tooling

`pytest + pytest-asyncio + pytest-cov + respx / unittest.mock.` **[TODO: Add type-checker (mypy / pyright) result.]**

Limitations & what we'd do next

Where the system is brittle today

- ▶ [TODO: e.g. no multi-provider failover; if Anthropic is down, we are.]
- ▶ [TODO: e.g. SQLite is single-writer; horizontal scaling needs Postgres.]
- ▶ [TODO: e.g. no rate-limit budget across processes (only within one).]

Given another week we would

1. [TODO: e.g. add OpenTelemetry traces around every external call.]
2. [TODO: e.g. load-test with k6 to find the real bottleneck.]
3. [TODO: e.g. ship a Streamlit UI on top of the same business logic.]

Team contributions

Member	Primary ownership	Commits
[TODO: Member A]	[TODO: e.g. service layer, retries, config, CI]	[TODO: 34%]
[TODO: Member B]	[TODO: e.g. HTTP API, concurrency, benchmark]	[TODO: 33%]
[TODO: Member C]	[TODO: e.g. storage, tests, Docker, report]	[TODO: 33%]

We can all answer:

Any of us can walk through any module on demand. Pick a file — we'll defend it.

AI tool use: [TODO: One-sentence disclosure (e.g. “Cursor scaffolded the FastAPI routes; we rewrote auth and validation.”).]

Questions?

[TODO: Team Name]

[TODO: `github.com/your-team/your-repo`]

“We can walk through any file in the repo.”